arXiv:2608.04968v1 [cs.LG] 5 Aug 2026

EVOLVENET: COLLABORATIVE HARNESS EVOLUTION FOR AGENT SELF-IMPROVEMENT

Jun Nie[1,2] Yonggang Zhang[3] Qianshu Cai[2,3]
Yiu-ming Cheung[1] Xinmei Tian[2] Bo Han[1]

[1]Hong Kong Baptist University
[2]University of Science and Technology of China
[3]The Hong Kong University of Science and Technology

Code: https://github.com/junnie00/EvolveNet

ABSTRACT

The capabilities of an LLM agent depend not only on its model but on the *harness*: the executable program that constructs context, invokes tools, verifies results, and recovers from failure. Recent work shows that evolving the harness yields persistent improvements without updating model weights. Existing approaches, however, assume that all execution experience can be routed to a single optimizer, which evolves one harness along a sequential trajectory. Real agent ecosystems violate that assumption: users, organizations, and environments generate isolated streams of experience that cannot be pooled, so the experience most worth learning from is exactly the experience that cannot be directly centralized. We introduce *EvolveNet*, a paradigm of collaborative harness evolution that moves experience extraction to the data. A shared harness is broadcast to data-local agent deployments, each of which evolves it on its own workload. Only the resulting program adaptations are composed into an updated shared harness and redistributed, so that every participating agent inherits operational experience discovered by the others. By shifting the aggregation boundary from raw workloads to learned adaptations, EvolveNet keeps workloads local and allows multiple evolutionary searches to proceed concurrently with reduced serial depth. Because independently modified programs cannot be averaged like model parameters and may conflict when composed, EvolveNet introduces scope-typed, evidence-guided program aggregation. Across five settings spanning text-to-SQL, data-science coding, competitive programming, software engineering, and agentic workflows, EvolveNet improves the shared harness in all five, with the largest gains under heterogeneous workloads, and ablations attribute the improvement to composition of adaptations from different agents rather than to selecting among them.

1 INTRODUCTION

Large language model agents are not defined by the model alone. Their behavior also depends critically on the surrounding *harness*: the executable program that constructs context, invokes models and tools, verifies intermediate results, and determines how the system responds to failure. Two agents built on the same frozen model can exhibit substantially different capabilities simply because their harnesses implement different reasoning procedures, tool-use policies, and recovery mechanisms. This observation has motivated a growing line of work on improving the program around the model, including prompt optimization, workflow search, and direct evolution of agent source code (Fernando et al., 2023; Yang et al., 2024; Hu et al., 2025; Zhang et al., 2025a). More recent studies make the harness itself the persistent state of adaptation, showing that an agent can acquire durable behavioral improvements without modifying its underlying model weights (He et al., 2025; Lee et al., 2026; Zhang et al., 2026; Nie et al., 2026).

Existing harness-evolution methods, however, largely follow a centralized paradigm. Workloads, execution traces, and behavioral feedback must first be made available to a single optimizer, which extracts useful experience from them and evolves one harness along a largely sequential trajectory. This paradigm is natural when all experience is centrally accessible, but it couples the exploitation of distributed experience to workload centralization. Moreover, as additional workloads are introduced, their adaptations must be explored through the same evolving program, limiting both the breadth of search and the serial efficiency of harness evolution.

Real agent deployments are naturally distributed across users, organizations, databases, repositories, and tool environments. This suggests reversing the order in which experience is extracted and aggregated. Rather than first pooling the workloads that contain useful experience and then learning from them centrally, each deployment can first translate its local executions and failures into an improved harness, after which the resulting program adaptations can be aggregated. In other words, the object crossing the system boundary changes from experience-bearing workloads to executable representations of the experience learned from them.

This shift creates a different paradigm for agent self-evolution. Local workloads can remain where they are generated, while multiple deployments explore distinct evolutionary trajectories concurrently. Their discoveries can then be accumulated in a shared harness and redistributed, so that each client begins the next round from adaptations the others discovered. Parallel local evolution does not reduce the total amount of search, but it reduces its serial depth: the client phase is governed by the slowest concurrent branch rather than by the sum of all local searches.

We introduce **EvolveNet** to realize this paradigm. EvolveNet begins each round from a common shared harness, broadcasts it to multiple data-local clients, and evolves it independently on their local workloads. Each client thereby extracts local operational experience into a specialist program. The server then integrates the returned adaptations into an updated shared harness and redistributes it as the shared starting point for the next round. EvolveNet therefore replaces the conventional *aggregate-data-then-evolve* pipeline with an *evolve-locally-then-aggregate* cycle, turning distributed local search into cumulative progress in one executable agent system.

The key obstacle is aggregation, and it is not the one federated optimization solves: the object being combined is neither a parameter vector nor a collection of modules with a predefined composition rule, but interacting executable programs whose semantics appear only when they run. In parameter-space aggregation, locally learned information is encoded in vectors that can be combined through arithmetic operations such as weighted averaging. Harness adaptations are source-program modifications: they have no meaningful arithmetic average, and individually beneficial edits may become redundant, contradictory, or invalid when composed. One client may strengthen a verification rule that another removes; two clients may alter the same control path for different local reasons; and a locally useful prompt edit may damage behavior elsewhere. Collaborative harness evolution therefore depends not merely on communication, but on determining which adaptations are transferable, how local specialization should be preserved, and whether their composition retains existing capabilities.

EvolveNet addresses this challenge through evidence-guided, scope-typed program aggregation. All clients branch from the same broadcast harness, allowing the server to reason over their program deltas relative to a common base. Each delta is accompanied by measured behavioral changes, enabling the server to integrate complementary mechanisms while distinguishing globally transferable behavior from adaptations that should remain domain-conditioned. Candidate merges are behaviorally validated before they become the next shared harness. Clients are encouraged to specialize in their own workload slices so that parallel searches produce complementary rather than redundant adaptations.

Rounds therefore form a bidirectional adaptation loop: local specialists and the shared harness shape one another: each specialist is evolved from the current shared program, while the next shared program is composed from the adaptations those specialists discover. Collaboration here is mediated by an artifact rather than interactive — clients neither communicate nor coordinate within a round; experience found by one becomes available to the others only through the redistributed harness.

Evaluating the aggregation operator requires separating its contribution from the substantial stochasticity and path dependence of local harness evolution. We therefore introduce a paired evaluation protocol in which competing aggregation rules are applied to identical snapshots of locally evolved

client programs. This isolates differences caused by aggregation from differences in the client populations received by each rule.

Across five settings spanning text-to-SQL, data-science coding, competitive programming, software engineering, and agentic workflows, EvolveNet improves the shared harness in every setting, with its largest gains when deployments hold disjoint forms of expertise. On the most heterogeneous setting, paired ablations attribute these gains to composing client adaptations rather than selecting or routing among complete specialists: the merged harness retains adaptations discovered by different clients and combines them into behavior that no single client produced. Our contributions are as follows:

- We introduce **collaborative harness evolution**, a paradigm that moves experience extraction from a centralized optimizer to data-local agent deployments and aggregates the resulting executable adaptations rather than the workloads that produced them. This enables distributed experience to accumulate in a shared harness while local evolutionary searches proceed concurrently (Sec. 4).
- We identify **program aggregation** as the central technical challenge of this paradigm and develop an evidence-guided, scope-typed aggregation operator that integrates harness deltas relative to a common base, promotes transferable mechanisms to global behavior, and preserves domain-specific adaptations under conditional scopes (Sec. 4.3).
- We establish an evaluation protocol that separates aggregation quality from the stochasticity of local evolution, and apply it across five settings. Complementary adaptations discovered by different clients accumulate in one shared harness, with the largest gains under heterogeneous workloads, while parallel local evolution reduces the serial depth of search and requires only one server-side aggregation session per round (Secs. 6.2–6.8).

2 Related Work

Optimizing the program around a frozen model. A substantial body of work improves LLM systems by optimizing the structures that surround the model rather than its weights. Prompt optimization searches for instructions through iterative refinement, evolutionary search, or by treating the LLM itself as an optimizer (Zhou et al., 2023; Fernando et al., 2023; Yang et al., 2024; Guo et al., 2024); DSPy optimizes multi-stage pipelines (Khattab et al., 2024); and automated agent design searches over tool-use workflows and control structures (Hu et al., 2025; Zhang et al., 2025b). A closely related line treats executable programs themselves as the object of improvement — STOP recursively improves code-generating programs (Zelikman et al., 2023), FunSearch pairs language models with program evaluation (Romera-Paredes et al., 2024), the Darwin Gödel Machine explores open-ended self-modification (Zhang et al., 2025a), and MOSS rewrites an agent's own source (Cai et al., 2026) — and, most recently, makes the agent *harness* the persistent state of adaptation, optimizing the control program that organizes model calls, tools, feedback and recovery (He et al., 2025; Lee et al., 2026; Zhang et al., 2026; Nie et al., 2026). These methods are typically instantiated as a single optimization process over centrally accessible experience, producing one evolutionary trajectory. They study how *one* agent improves; EvolveNet studies how improvements discovered by *many* agents accumulate in a shared evolving artifact. EvolveNet asks what changes when that experience is partitioned across deployments and cannot be centralized.

Collaborative improvement through shared artifacts. A line of work enables distributed agents to collaborate by exchanging learned artifacts rather than the data that produced them. FederatedSkill shares semantic skill diffs that a server folds into a shared skill library (Yang et al., 2026), Fed-SE aggregates parameter-efficient adapter updates across environments (Chen et al., 2025), and Federation over Text distills clients' reasoning traces into a shared textual insight library (Yao et al., 2026). All inherit the broadcast–optimize–aggregate–redistribute schedule of decentralized model training, from FedAvg (McMahan et al., 2017) through methods addressing instability and statistical heterogeneity (Li et al., 2020; Karimireddy et al., 2020; Reddi et al., 2021). EvolveNet shares the high-level communication pattern of these systems, but the object it optimizes and the aggregation problem that object induces are of a different kind. Each of them assumes client updates can be expressed in a predefined aggregation space — a weight tensor, an adapter, a list of skills, a set of textual insights — in which combination is well defined a priori. Here the aggregation space is itself a programming language, and whether two contributions combine is a fact about execution rather

than a property of the representation. What it circulates is the *executable control program* itself, and targets a single scope-conditioned shared artifact rather than a library of independent patches, insights, or adapter weights. This changes what aggregation is: skill patches compose by insertion and insights by concatenation, whereas program edits interact through control flow, prompts, tool calls, and runtime state, so the server cannot rely on averaging, numerical distance, or any gradient-based notion of agreement, and must instead reason about program behavior.

A different sense of collaboration appears at inference time, where agents exchange messages, debate, or delegate subtasks in order to answer one query (Du et al., 2024; Wu et al., 2023). EvolveNet collaborates across adaptation rounds instead: its clients evolve independently within a round and never exchange messages, and their contributions meet only in a persistent shared harness that accumulates and redistributes them.

Model merging. Model merging consolidates independently acquired capabilities relative to a common base without retraining on the union of their data. Task Arithmetic composes fine-tuned models as parameter displacements (Ilharco et al., 2023), TIES-Merging reduces interference by removing weak updates and resolving sign conflicts (Yadav et al., 2023), and DARE sparsifies and rescales deltas before merging (Yu et al., 2024). EvolveNet shares the goal of combining capabilities relative to a shared base while managing interference, but the aggregation spaces differ fundamentally: neural checkpoints admit coordinate-wise arithmetic, whereas source-code edits have no intrinsic magnitude, direction, or element-wise correspondence, and syntactically valid edits can interact in ways not inferable from the edits alone. EvolveNet therefore uses execution-derived behavioral evidence both to guide integration and to validate it, and aims not merely to select among complete client programs but to preserve and combine the transferable adaptations distributed across them.

3 PRELIMINARIES

Harness. Let M be a frozen LLM. A harness h is an executable program that, given an input x, may call M any number of times, execute tools, inspect results, and return an answer $h(x)$. The space $\mathcal{H}$ of harnesses is the space of programs satisfying two invariants (Sec. 5): the model is never changed, and gold labels are never read at inference.

Our local-evolution loop instantiates the trace-driven harness evolution of TTHE (Nie et al., 2026), with an LLM proposer that reads execution traces and edits the harness source.

Distributed setting. We consider K agent deployments that are *data-local*: each runs where its workload is generated, and evolution happens there rather than at a central site. We refer to them throughout the protocol as *clients*. Client k holds a workload D_k that cannot be pooled with the others. Client k can execute a harness on D_k, observe traces, and grade outcomes on its own data. Raw workloads and execution records stay local; what crosses the boundary is program text, its delta against the common base, and per-item behavioral verdicts.

Objective. We seek a single shared harness $h^\star$ maximizing expected accuracy on a target workload $\mathcal{P}_{\text{target}}$, using only local evolution at the clients and aggregation at the server. Only the client mixture is accessible during evolution; the target may also contain related domains held by no client, as one of our evaluation domains does.

4 METHOD: EVOLVENET

Let M denote a frozen language model and let a harness $h \in \mathcal{H}$ be the executable program that organizes calls to M, invokes tools, constructs context, verifies intermediate results, and handles failures. We consider K clients, where client k holds a local workload D_k^{tr} drawn from deployment domain d_k. Raw workloads remain at their respective clients. Our objective is to construct a single shared harness that accumulates useful adaptations discovered across these domains:

$$h^\star = \arg\max_{h \in \mathcal{H}} \mathbb{E}_{z \sim \mathcal{P}_{\text{target}}} \left[s(h; z) \right], \tag{1}$$

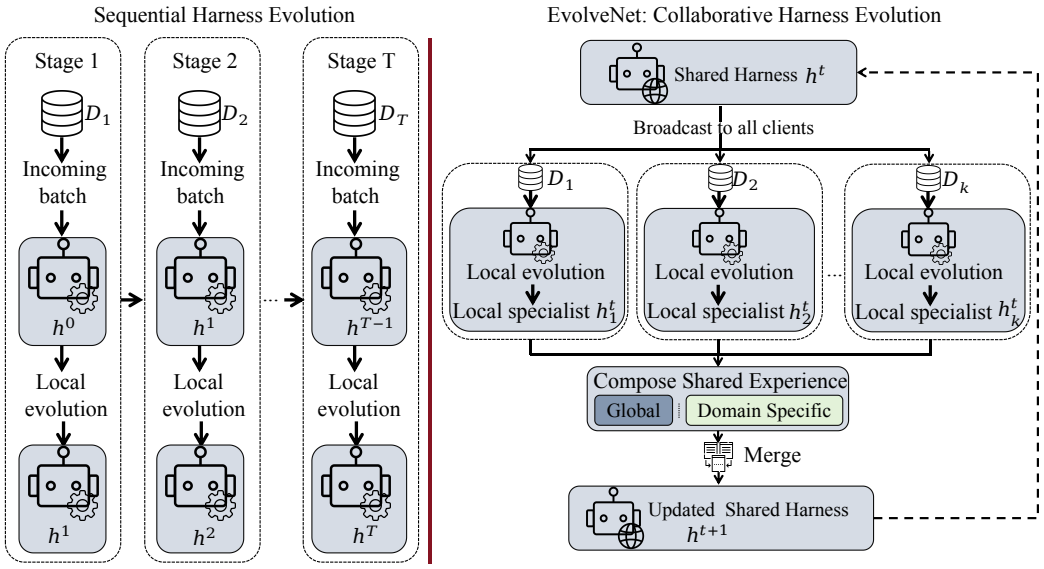


Figure 1: **Left:** conventional harness evolution is sequential. One optimizer consumes incoming batches in order and each harness must be produced before the next can be explored, so a T-stage search has serial depth T and follows a single trajectory. **Right:** EvolveNet broadcasts the shared harness $h^{(t)}$ to K clients, which evolve it in parallel on the workloads held at each, into specialists $h_k^{(t)}$. The server aggregates the returned programs rather than the workloads behind them, adopting a mechanism globally when its supporting evidence spans domains and conditioning it on its home domain otherwise, and commits the result as $h^{(t+1)}$ only if it passes the per-item acceptance test (Sec. 4.4). The dashed edge closes the loop: the committed harness is redistributed, so each client's next round of evolution starts from what the whole collaboration has learned, not from its own previous program.

where $\mathcal{P}_{\text{target}}$ is the deployment distribution the shared harness must serve and s is a task-specific evaluation function. During evolution EvolveNet has access only to the client mixture $\mathcal{P}_{\text{client}} = \sum_{k=1}^{K} \pi_k \mathcal{P}_k$, where $\mathcal{P}_k$ is client k's deployment distribution and π_k its mixture weight; $\mathcal{P}_{\text{client}}$ may cover only part of $\mathcal{P}_{\text{target}}$, and our evaluation includes a domain held by no client (Table 2). The model M remains fixed throughout; all adaptation occurs in the harness.

4.1 From centralized evolution to collaborative harness evolution

Conventional harness evolution follows an *aggregate-workloads-then-evolve* paradigm. Workloads, execution traces, and behavioral feedback are first made available to a centralized optimizer, which extracts useful experience from them and updates one harness along a single evolutionary trajectory. In conceptual form,

$$\underbrace{\text{Pool}\left(D_1^{\text{tr}}, \ldots, D_K^{\text{tr}}\right) \longrightarrow \text{Evolve}}_{\text{centralized harness evolution}}. \tag{2}$$

This design couples experience extraction to workload centralization. It also places all adaptation pressure on one evolving program: additional workloads can enlarge the available experience, but their adaptations must still be discovered through the same central search process.

EvolveNet reverses this order (Fig. 1). Each client first extracts operational experience from its own workload by evolving the shared harness locally. The server then aggregates the resulting *program adaptations*, rather than the workloads that produced them:

$$\underbrace{\text{Evolve}_{1:K}\left(h^{(t)}, D_{1:K}^{\text{tr}}\right) \longrightarrow \text{Aggregate}\left(\Delta_{1:K}^{(t)}\right)}_{\text{collaborative harness evolution}}. \tag{3}$$

At round t, all clients receive the same shared harness $h^{(t)}$. Client k applies a local evolution operator $\mathcal{E}$ to obtain

$$h_k^{(t)} = \mathcal{E}\left(h^{(t)}, D_k^{\mathrm{tr}}\right). \tag{4}$$

The local proposer is encouraged to specialize in the recurring structures and failure modes of domain d_k, rather than reproduce a generic solution already available to all clients. The broadcast harness is always retained in the local candidate pool. Therefore, if S_k denotes the client-side selection score,

$$S_k\left(h_k^{(t)}\right) \geq S_k\left(h^{(t)}\right), \tag{5}$$

up to the resolution of the client evaluator. Local specialization thus produces candidate capabilities without forcing a client to return a program that is measurably worse than the shared starting point on its own workload.

The server integrates the returned adaptations into a candidate shared harness,

$$\widetilde{h}^{(t+1)} = \mathcal{A}\left(h^{(t)}, h_1^{(t)}, \ldots, h_K^{(t)}\right), \tag{6}$$

which, after behavioral validation, is redistributed as the starting point for the next round. This produces a recurring SPECIALIZE $\rightarrow$ AGGREGATE $\rightarrow$ REDISTRIBUTE cycle. Adaptations discovered locally are therefore not terminal client-specific outcomes: once integrated into the shared harness, they become part of the initial state from which all clients evolve in the following round. This is the sense in which the local specialists and the shared harness co-evolve — each specialist is evolved from the current shared program, while the next shared program is assembled from the adaptations those specialists produce. Clients do not interact directly within a round; their coupling occurs through the shared executable artifact across rounds.

This change in paradigm also alters the serial structure of search. Let $C_k^{(t)}$ be the latency of the local search conducted by client k in round t, and let $C_{\mathcal{A}}^{(t)}$ and $C_{\mathrm{val}}^{(t)}$ denote aggregation and validation latencies. Executing the same K local searches serially would require $C_{\mathrm{serial}}^{(t)} = \sum_k C_k^{(t)} + C_{\mathcal{A}}^{(t)} + C_{\mathrm{val}}^{(t)}$, whereas the parallel client phase in EvolveNet has serial depth

$$C_{\mathrm{EvolveNet}}^{(t)} = \max_k C_k^{(t)} + C_{\mathcal{A}}^{(t)} + C_{\mathrm{val}}^{(t)}. \tag{7}$$

EvolveNet does not eliminate the computational cost of local search; it makes multiple evolutionary trajectories available at a serial depth governed by the slowest client rather than by their sum.

4.2 THE PROGRAM-AGGREGATION CHALLENGE

The paradigm above depends on an aggregation operation that has no direct counterpart in parameter-space aggregation. Model updates inhabit a common vector space and can be combined by coordinate-wise arithmetic, as in weight averaging (McMahan et al., 2017). Source programs do not. There is no meaningful average of two Python harnesses, and independently useful modifications need not remain useful when placed in the same executable system.

Program aggregation must address three coupled difficulties. First, it must establish *attribution*: a returned program may contain several interacting edits, and the server must determine which behavioral changes are associated with that client variant. Second, it must manage *interference*: two adaptations may be redundant, may alter the same control path in incompatible ways, or may encode different assumptions about the deployment domain. Third, it must control *regression*: syntactically valid and individually successful edits may still break capabilities already present in the shared harness after composition.

EvolveNet addresses these difficulties through four connected principles: all clients are compared against a common program base; source-level changes are accompanied by measured behavioral evidence; candidate mechanisms are assigned an explicit deployment scope before composition; and the composed program is evaluated behaviorally before it replaces the current shared harness.

4.3 EVIDENCE-GUIDED, SCOPE-TYPED PROGRAM AGGREGATION

Common-base program deltas. Because every client starts from the same $h^{(t)}$, its contribution can be represented as

$$\Delta_k^{(t)} = \mathrm{diff}\left(h^{(t)}, h_k^{(t)}\right). \tag{8}$$

The server edits a byte-for-byte copy of $h^{(t)}$ rather than synthesizing a new program from K complete files. This common-base representation serves two purposes. It makes client contributions directly comparable, since each delta expresses what changed relative to the same behavioral state, and it limits uncontrolled artifact growth caused by concatenating entire client programs.

The aggregator decomposes each delta into candidate *mechanisms*. A mechanism is a semantically coherent adaptation that implements a recognizable behavior — for example, an execution-retry rule, a schema probe, an output-format constraint, or a repository-specific repair procedure. A mechanism need not correspond exactly to one syntactic diff hunk: several nearby edits may jointly implement one behavior.

Behavioral evidence. Source code alone does not reveal whether a client modification is useful. While selecting its local program, client k therefore compares $h_k^{(t)}$ with the broadcast harness on its own workload and reports the resulting behavioral changes. Let $v_i(h) \in \{0, 1\}$ denote the task evaluator's verdict for item i. We define

$$F_k^{(t)} = \left\{ i \in D_k^{\mathrm{tr}} : v_i\left(h^{(t)}\right) = 0,\ v_i\left(h_k^{(t)}\right) = 1 \right\}, \tag{9}$$

$$B_k^{(t)} = \left\{ i \in D_k^{\mathrm{tr}} : v_i\left(h^{(t)}\right) = 1,\ v_i\left(h_k^{(t)}\right) = 0 \right\}, \tag{10}$$

and attach $e_k^{(t)} = (F_k^{(t)}, B_k^{(t)})$ to the corresponding program delta. The evidence records which capabilities were newly acquired and which were lost relative to the common base. It does not by itself establish a causal mapping from each item to a particular line of code; rather, it constrains the aggregator's interpretation of the delta with observed behavior. The aggregation operator can now be written more precisely as

$$\widetilde{h}^{(t+1)} = \mathcal{A}\left(h^{(t)}, \left\{\left(\Delta_k^{(t)}, e_k^{(t)}\right)\right\}_{k=1}^{K}\right). \tag{11}$$

The adoption problem. Aggregation is not a free-form rewrite. Decomposing the K deltas yields a candidate set $\mathcal{M} = \{m_1, \ldots, m_n\}$, each m_j carrying the evidence of the client it came from. An *adoption* is a pair (S, τ): a subset $S \subseteq \mathcal{M}$ to incorporate and a scope assignment τ for its members, which together determine one composed program $h(S, \tau)$. Writing $\mathrm{Fix}(S, \tau)$ and $\mathrm{Brk}(S, \tau)$ for the items of the server's validation slice D^{val} that $h(S, \tau)$ newly solves and newly fails relative to $h^{(t)}$, the server seeks

$$\max_{S \subseteq \mathcal{M},\, \tau} \ |\mathrm{Fix}(S, \tau)| \quad \text{subject to} \quad |\mathrm{Brk}(S, \tau)| \leq |\mathrm{Fix}(S, \tau)|. \tag{12}$$

This is the aggregation problem in program space, and it is not solvable by search: the objective and the constraint are defined by *executing* $h(S, \tau)$, so each of the $2^{|\mathcal{M}|}$ subsets and their scope assignments would cost a full evaluation pass, and the mechanisms interact, so the objective is neither separable nor monotone in S. EvolveNet therefore splits Eq. 12 into a proposal step and an exact feasibility check. The scope-typed aggregator (Sec. 4.3) proposes one (S, τ) in a single pass by reasoning over the attached evidence rather than by enumeration; the acceptance gate (Sec. 4.4) then evaluates the realized program and enforces the constraint exactly, rejecting the proposal when it fails. What the aggregator supplies is a heuristic for *which* mechanisms to adopt and *where* they should apply; what the gate supplies is the guarantee that an adopted proposal actually satisfies Eq. 12.

Scope-typed composition. The central decision is not simply whether to accept or reject a mechanism, but *where* that mechanism should apply. Before composition, every candidate mechanism m receives a scope

$$\tau(m) \in \{\textsc{global}\} \cup \{\textsc{home}(d_k)\}_{k=1}^{K}. \tag{13}$$

Writing $\mathrm{ev}(m)$ for the items credited to m, the assignment follows one rule:

$$\tau(m) = \begin{cases} \text{GLOBAL} & \text{if } \mathrm{ev}(m) \text{ meets two or more domains, or } m \text{ addresses a domain-independent failure mode,} \\ \text{HOME}(d_k) & \text{if } \mathrm{ev}(m) \subseteq D_k^{\mathrm{tr}} \text{ for exactly one } k, \\ \bot \text{ (reject)} & \text{if } \mathrm{ev}(m) \text{ is a single item.} \end{cases} \tag{14}$$

Domain-independent failure modes are those the evidence cannot localize even in principle — malformed output, execution failure, a generally applicable recovery procedure. A GLOBAL mechanism enters as one implementation applied to every input; a HOME(d_k) mechanism enters conditionally, activated only when the harness observes that the current input belongs to d_k; and the last case removes adaptations that encode one problem's quirk rather than a recurring class.

This distinction changes the structure of program conflicts. Suppose two clients modify the same behavior in incompatible ways, but the evidence for each modification is confined to a different domain. A parameter-space aggregator must either average the updates or choose one. EvolveNet can retain both adaptations under disjoint domain conditions, so that many apparent conflicts become conditional composition rather than a winner-take-all decision.

This property is not unconditional; what it requires can be stated exactly. Let $\kappa(x)$ denote the dispatch key the harness observes for input x.

Proposition 1 (Scope isolation)**.** *Let m be adopted with scope* HOME(d) *and composed so that every statement it contributes is reachable only under a guard testing $\kappa(x) = d$. If m writes no state read outside that guard and alters no prompt text applied outside it, then for every input x with $\kappa(x) \neq d$, $h(S \cup \{m\}, \tau)(x) = h(S, \tau)(x)$.*

Proposition 2 (Conflicts under disjoint scopes)**.** *Let m_i and m_j satisfy the hypothesis of Prop. 1 with scopes* HOME(d_i) *and* HOME(d_j)*, $d_i \neq d_j$. Then $h(S \cup \{m_i, m_j\}, \tau)$ agrees with $h(S \cup \{m_i\}, \tau)$ on every input of domain d_i and with $h(S \cup \{m_j\}, \tau)$ on every input of domain d_j, whatever m_i and m_j do to the same behavior.*

Both follow from the same observation: under the hypothesis the statements of a home-scoped mechanism are unreachable off its domain and no object read along that execution has been altered, so the two programs induce the same trace. Prop. 2 is what licenses the treatment of conflicts described above: two clients that changed the same behavior in opposite directions need not be arbitrated at all if their evidence lies in different domains, because each side is unreachable where the other applies. A parameter-space aggregator has no counterpart of this move — coordinate-wise combination cannot make an update conditional on the input it will later be asked about.

The value of these propositions lies in making the required isolation assumptions explicit rather than in the proofs themselves. They state exactly what an implementation must respect for domain conditioning to bound interference by construction, and they are why the aggregator introduces home-scoped material as guarded extensions rather than as edits to shared prompt structure. Where the hypothesis fails — a mechanism that touches shared state despite its guard — isolation does not hold, and the constraint of Eq. 12 is what detects it. Conflicts involving genuinely shared behavior, or mechanisms that interact through common state, still require the aggregator to choose a single side. For a mechanism that is adopted under a home scope, the server prefers preserving the client's verified implementation rather than paraphrasing it, since source-level rewriting may remove the behavior that produced the measured gain.

The adoption procedure. Eq. 12 is approached in five steps. (i) Diff each $h_k^{(t)}$ against $h^{(t)}$ to isolate what client k added. (ii) Read the execution traces attached to that delta. (iii) Group the edits into mechanisms and attach to each the items of $F_k^{(t)}$ it is credited with. (iv) Classify every mechanism by Eq. 14. (v) Compose: one implementation per GLOBAL mechanism, each HOME(d) mechanism copied verbatim into a branch guarded on d, and conflicting mechanisms with disjoint home domains both retained — only GLOBAL conflicts are arbitrated, by graded evidence.

Steps (i)–(iii) are mechanical. Steps (iv) and (v) are carried out by an LLM merger applying the stated criteria to the attached evidence, because deciding whether a mechanism's evidence is localizable, and how to place it without disturbing the base, requires reading the program rather than scoring it. That division is what makes the operator testable: step (iv) is the only part the aggregation

variants of Sec. 6.4 change, so their differences are attributable to the classification rule alone, and step (v)'s output is checked exactly against Eq. 12 before anything is committed.

Preserving the shared base. The shared harness is not treated as disposable source text. It represents the behavioral state accumulated through previous rounds and therefore acts as a set of load-bearing invariants during aggregation. This is particularly important for prompt rules: unlike a conditional code path, an existing prompt instruction may influence every input. Rewriting or "consolidating" such a rule can introduce system-wide changes unrelated to the mechanism being adopted. Accordingly, the aggregator modifies the broadcast harness in place, preserves existing prompt rules, and integrates new mechanisms as localized extensions whenever possible. The implementation also constrains growth relative to the base harness; exact budget settings are reported with the experimental configuration. These restrictions do not make composition risk-free, but they reduce the number of unrelated behaviors changed by a single merge.

4.4 Behavioral commitment and rollback

Program composition may introduce regressions that cannot be determined reliably from the source diffs alone. EvolveNet therefore separates *construction* of a candidate shared harness from *commitment* to that candidate. Let D^{val} be a validation slice that is disjoint from the client workloads used to produce the candidate and from the final test set. Relative to the previous shared harness $h^{(t)}$, define

$$F_{\mathrm{val}} = \left\{ i \in D^{\mathrm{val}} : v_i\left(h^{(t)}\right) = 0,\ v_i\left(\widetilde{h}^{(t+1)}\right) = 1 \right\}, \tag{15}$$

$$B_{\mathrm{val}} = \left\{ i \in D^{\mathrm{val}} : v_i\left(h^{(t)}\right) = 1,\ v_i\left(\widetilde{h}^{(t+1)}\right) = 0 \right\}. \tag{16}$$

The candidate is committed iff

$$|F_{\mathrm{val}}| \geq |B_{\mathrm{val}}|\,. \tag{17}$$

The gate compares behavioral transitions item by item rather than subtracting aggregate scores. Most items are unchanged between two related harnesses; focusing on the items that flip makes the decision depend on the behavioral difference introduced by the merge.

When the candidate fails the test, the aggregator receives the observed regressions and is allowed one revision. If the revised candidate also fails, the round rolls back to $h^{(t)}$. Rollback always targets the previous shared harness, not the best-performing client, preserving the invariant that every round begins from one shared program state. The gate protects the committed validation trajectory under the criterion in Eq. 17; it does not guarantee improvement on an unseen test distribution. Its role is narrower: to prevent an aggregation whose measured regressions outnumber its measured gains from becoming the shared starting point of all subsequent local searches. Because the test consumes only per-item verdicts, it can also be computed without centralizing validation items: with the slice partitioned across clients, each returns two integers and the server sums them, reproducing the same accept/reject decision. The revision step is different — it shows the aggregator which items regressed — so the two-integer variant supports gating but not revision.

4.5 The complete EvolveNet procedure

Algorithm 1 states one round. The server broadcasts $h^{(t)}$ to all K clients. Each client independently evolves it on its local workload and returns a specialist harness $h_k^{(t)}$, its common-base delta, and its behavioral evidence. The server decomposes the deltas into candidate mechanisms, assigns each a global or home-domain scope, and composes them into $\widetilde{h}^{(t+1)}$. The candidate is compared with $h^{(t)}$ on D^{val} and committed only if it passes Eq. 17; otherwise it is revised once and, if still unsuccessful, discarded.

5 Experimental Setup

We evaluate EvolveNet on five settings with a natural axis of agent specialization; each unit of specialization is one data-local client in the protocol. Clients are databases on BIRD text-to-SQL (Li et al., 2023), libraries on DS-1000 (Lai et al., 2023), repositories on SWE-bench Verified (Jimenez

Algorithm 1 EvolveNet: one communication round

input: shared harness $h^{(t)}$; client workloads $D^{\mathrm{tr}}_{1:K}$; validation slice D^{val}; local search budget (E, G)
for $k = 1$ **to** K **in parallel do**
  $h_k^{(t)} \leftarrow \textsc{LocalEvolve}\left(h^{(t)}, D_k^{\mathrm{tr}}, E, G\right)$
  $\Delta_k^{(t)} \leftarrow \mathrm{diff}\left(h^{(t)}, h_k^{(t)}\right)$
  $e_k^{(t)} \leftarrow \textsc{BehaviorChanges}\left(h_k^{(t)}, h^{(t)}; D_k^{\mathrm{tr}}\right)$
end for
$\widetilde{h}^{(t+1)} \leftarrow \textsc{ScopeAggregate}\left(h^{(t)}, \left\{\left(\Delta_k^{(t)}, e_k^{(t)}\right)\right\}_{k=1}^{K}\right)$
$(F_{\mathrm{val}}, B_{\mathrm{val}}) \leftarrow \textsc{Compare}\left(\widetilde{h}^{(t+1)}, h^{(t)}; D^{\mathrm{val}}\right)$
if $|B_{\mathrm{val}}| > |F_{\mathrm{val}}|$ **then**
  $\widetilde{h}^{(t+1)} \leftarrow \textsc{ReviseAggregate}\left(h^{(t)}, \left\{\left(\Delta_k^{(t)}, e_k^{(t)}\right)\right\}_{k=1}^{K}, B_{\mathrm{val}}\right)$
  recompute $(F_{\mathrm{val}}, B_{\mathrm{val}})$
end if
$h^{(t+1)} \leftarrow \widetilde{h}^{(t+1)}$ **if** $|F_{\mathrm{val}}| \geq |B_{\mathrm{val}}|$ **else** $h^{(t)}$
return $h^{(t+1)}$

et al., 2024), task families on ClawEval (agentic workflow tasks over an OpenClaw agent with mock enterprise services), and difficulty bands on LiveCodeBench (Jain et al., 2025), which serves as a smaller robustness check since its single-platform pool admits no five-way sharding. Weights π_k in the accessible client mixture $\mathcal{P}_{\mathrm{client}}$ are uniform, and each client's proposer is given a search role — conservative repair, independent exploration, or adversarial audit — assigned by client index so that concurrent branches do not collapse onto the same edit; Appendix C reproduces the specialist instruction and the aggregator's adoption rule verbatim. We use $K{=}5$ clients everywhere except LiveCodeBench ($K{=}3$), and $T{=}3$ rounds throughout, with one local generation and one proposer branch per round ($E{=}G{=}1$ in Alg. 1). Held-out sets contain 150 (BIRD), 200 (DS-1000), 40 (SWE-bench) and 30 items (ClawEval, LiveCodeBench); splits are fixed with seed 0 and every item passed a gold check in our environment before use (Appendix A). Appendix B gives the per-client shards and split sizes.

The frozen solver is deepseek-v4-flash throughout, and the same model drives the client proposers and the server's aggregator; Sec. 6.7 repeats the pipeline with a different stack. Only the harness adapts: a static auditor rejects any candidate that changes the model, endpoint, or credentials, or that reads a gold answer at inference. Labels enter the loop at two points only — clients select candidates on their own shard, the server gates merges on its validation slice — and test sets are evaluated after the shared harness is frozen. BIRD served as the development benchmark, on which the aggregation operator, the gate, and the loop design were selected; the operator was then frozen and the remaining four settings were each run end-to-end once. DS-1000 was designated in advance as the primary confirmation benchmark, being the setting whose clients hold the most disjoint workloads. Solver replies are cached on the full request and every child process runs with a pinned hash seed, so that a score difference between two harnesses reflects a behavioral difference rather than sampling noise (Appendix D).

6 Results

6.1 How much does a run vary?

Every comparison below is calibrated against this measurement, which we therefore establish first. We have three independent executions of the identical BIRD round-0 protocol (same shards, same specialist instruction, same budget; only sampling differs). Client-level validation scores vary by 2–4 items across these runs: the same mixed-shard client scores 66% and 69% on the validation slice in two runs, the formula_1 specialist 62% and 66%, the toxicology specialist 62% and

Table 1: Held-out accuracy (%) of the shared harness before evolution and after T EvolveNet rounds — whether experience extracted locally and aggregated as program adaptations becomes cumulative improvement in the common starting program. ClawEval is graded on a continuous scale, so its column reports the mean task score.

	BIRD	DS-1000	LCB	SWE-V	ClawEval
Unevolved harness	57.3	55.5	33.3	37.5	65.8
EvolveNet	**70.7**	**68.5**	**66.7**	**57.5**	**74.1**

65%; whole clients oscillate between returning an evolved program and returning the broadcast unchanged. Held-out scores of end-to-end finals produced by the same rule vary comparably (e.g. the select-best final scores 66.7% and 69.3% across two independent runs).

This spread sets the resolution of every comparison that follows. Differences of a few items fall inside it, so harnesses cannot be ranked by their totals. For each realized pair of harnesses we therefore report paired per-item outcomes rather than relying on aggregate score differences: the unit of evidence is a behavioral flip on a specific problem, not a shift in a sum.

The same spread separates the two roles a validation slice can play. As a gate it is sound: an accept or reject decision turns on which items flip, and flips are visible above the noise. As a ranking signal it is not — on DS-1000 the client with the highest validation total is nine points worse on held-out data than a merged harness with a lower one (Sec. 6.3). EvolveNet reads the slice only for per-item decisions, never to declare a winner.

6.2 Main results

Table 1 reports the held-out accuracy of the shared harness after T EvolveNet rounds, against the unevolved harness from which every client starts.

Collaborative evolution improves the shared harness on every setting, by 13.4 points on BIRD, 13.0 on DS-1000, 33.4 on LiveCodeBench, 20.0 on SWE-bench and 8.3 on ClawEval. Each improvement is significant under a paired test on that benchmark's held-out set — an exact McNemar test on binary outcomes, and a Wilcoxon signed-rank test on ClawEval's continuous judge scores — with p of 5.4e-4 (BIRD, win 26 / lose 6), 6.2e-6 (DS-1000, win 30 / lose 4), 2.0e-3 (LiveCodeBench, win 10 / lose 0), 0.021 (SWE-bench, win 9 / lose 1) and 0.022 (ClawEval, better 18 / worse 7); all five survive Holm correction for the five comparisons. EvolveNet also improves on retaining the strongest single client in all five settings (Appendix E), by margins that widen as the clients' workloads diverge — from 1.4 points on BIRD, whose eleven databases share one task shape, to 23.4 on LiveCodeBench.

Two properties of the trajectory are worth noting. The gain is not delivered in one step (Fig. 2): on DS-1000 the merged program goes from 60.0% after one round to 68.5% after three, with items broken by one round's merge recovered by the next, and on LiveCodeBench and SWE-bench the final round adds a further 16.7 and 17.5 points on held-out data over the artifact that preceded it. Nor is it confined to the libraries clients trained on: on DS-1000 the merged harness improves over the unevolved harness on every library (Table 2), including one that no client trained on. The remainder of this section asks what the aggregation operator contributes to these numbers (Sec. 6.4), whether the adaptations clients discover are actually retained (Sec. 6.5), and what the merged programs contain (Sec. 6.9).

6.3 Validation score does not rank programs — which is why the gate only gates

The server's validation slice is essential for gating and selection, but it cannot be used to *conclude*. On DS-1000 the select-best client scored 69% on validation but 59.5% on test, while the merged harness scored 63% on validation and 68.5% on test. Ranking by validation would invert the true ordering — consistent with the variance measurement of Sec. 6.1. EvolveNet therefore uses validation only for per-item accept/reject decisions (which are robust: they compare behavior flips, not

Table 2: DS-1000 held-out accuracy by library (%). *Delegation*, *GLOBAL-only* and *EvolveNet* are built from one identical final-round client snapshot; *select-best* is the end-to-end baseline run, in which the server promotes the client scoring highest on validation instead of aggregating. *Delegation* keeps all K client programs and dispatches each item to its library's owner; *GLOBAL-only* is EvolveNet's aggregator with domain conditioning forbidden (Sec. 6.4). Five libraries are covered by a client; Pytorch appears only in validation and test, so no specialist exists for it. Bold marks the EvolveNet column.

Library	n	Unevolved	Select-best	Delegation	GLOBAL-only	**EvolveNet**
Pandas	63	57.1	65.1	74.6	73.0	**73.0**
Numpy	47	51.1	55.3	55.3	61.7	**61.7**
Matplotlib	30	76.7	63.3	60.0	80.0	**83.3**
Sklearn	24	45.8	54.2	62.5	45.8	58.3
Scipy	21	42.9	57.1	57.1	61.9	**61.9**
no client trained on this library						
Pytorch	15	53.3	53.3	66.7	66.7	**66.7**
All	200	55.5	59.5	64.0	66.5	**68.5**

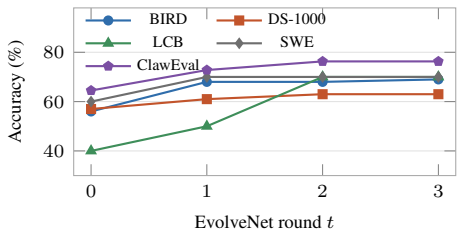


Figure 2: The committed shared harness over rounds, on each benchmark's validation slice (t=0 is the unevolved harness). The gate keeps the committed trajectory non-decreasing — a rejected merge leaves the curve flat rather than falling — provided the slice is fixed and scoring is deterministic (Appendix D). Scores are percentages of the slice maximum; ClawEval's is a mean judge score on the same axis.

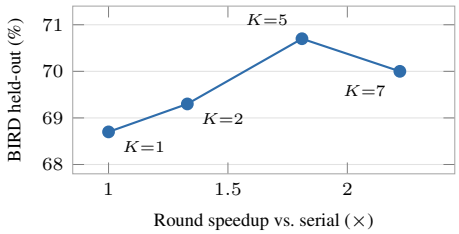


Figure 3: BIRD accuracy against how much faster an EvolveNet round completes than the same local searches executed serially, counting the non-parallelizable aggregation and gating steps; the client phase alone parallelizes 1.9–5.4× (Sec. 6.8). Splitting a fixed workload buys wall-clock without costing accuracy: every multi-client configuration is at or above K=1. The decline at K=7 is a shard-size effect: splitting the mixed shard by database leaves its three sub-clients with only 6–8 items each, and their candidates overfit them.

totals), and all conclusions in this paper are drawn from held-out test sets with paired significance tests.

6.4 Is this aggregation, or routing in disguise?

A scope-typed merge contains domain-conditioned material, which invites an alternative explanation: that EvolveNet simply preserves K specialists and dispatches by domain, and that no genuine composition occurs. We test this on DS-1000, the setting whose clients hold the most disjoint workloads, by replaying alternative aggregators on the *same* final-round client snapshot (Table 3). The most important comparison is against *select-best*, which performs no aggregation at all and simply keeps whichever client scores highest on the server's validation slice — the obvious thing to do if program composition were unnecessary.

The routing-only explanation does not survive. Composing the clients' mechanisms into one program beats keeping them apart and dispatching (68.5 vs. 64.0%, win 15 / lose 6), and — more telling — an aggregator *forbidden* from writing any domain condition already reaches 66.5%, above delegation. EvolveNet's gain is therefore driven mainly by mechanisms lifted into globally shared

Table 3: Do the two halves of the operator each earn their place? *Delegation*, *GLOBAL-only* and *EvolveNet* are built from one identical snapshot of DS-1000 client harnesses and evaluated on the same held-out set, so differences among them are attributable to the aggregation rule alone; *select-best* is the end-to-end baseline protocol, a run in which the server promotes the strongest client instead of aggregating. *Delegation* performs no composition at all: it keeps all K client programs and dispatches each item to its library's owner. *GLOBAL-only* is EvolveNet with domain conditioning forbidden — the aggregator may promote a mechanism only on cross-library evidence.

Aggregation rule	Composition	Domain gating	Held-out (%)
Unevolved harness	—	—	55.5
Select-best client (no aggregation)	—	—	59.5
Delegation (keep K programs, dispatch)	no	yes	64.0
GLOBAL-only merge	yes	no	66.5
EvolveNet (scope-typed merge)	yes	yes	**68.5**

behavior, with domain conditioning adding a further 2.0 points on top rather than carrying the result. The monotone ordering is consistent with both components contributing.

6.5 Does experience accumulate across agents?

The central promise of collaborative harness evolution is that an adaptation discovered by one agent becomes available to the others through the redistributed shared harness. Aggregate accuracy does not establish that this transfer occurs, so we measure composition directly. Let each client's *gain* be the held-out items it newly solves relative to the common harness broadcast to it that round, so that prior rounds' accumulated capability is not credited to the current clients. On DS-1000 the five clients' gains union to 20 items — of which the merged harness retains 18, a retention rate of 90.0%. Within that union, nine items are solved by the Pandas client alone, six by Sklearn alone and two by Matplotlib alone, so retention is not an artifact of clients agreeing: the merged program carries capabilities that exist in exactly one of its parents. Two further items are solved by the merged harness and by *no* individual client, i.e. composition produced behavior that no single trajectory reached.

A second observation on this benchmark points the same way. Pytorch appears only in validation and test, so no shard contains it and no specialist exists for it; the unevolved harness and select-best each solve 8 of its 15 items, while EvolveNet solves 10. Whatever produces that difference was not learned on Pytorch, which corroborates the mechanism-level reading above: part of what the merged program carries is behavior lifted out of the domain in which it was discovered.

6.6 How does EvolveNet compare with centralized evolution?

We give the centralized alternative two budgets. At *equal serial depth* (T=3 optimizer rounds, one proposer per round), the per-round-gated centralized optimizer over the pooled 100-item workload reaches 68.7% on the BIRD held-out; chaining its generations without per-round selection fails outright (the final candidate scores below the unevolved harness on its own training set and is rolled back). At *equal total budget* — a centralized *population* of five parallel proposer branches per round, matching EvolveNet's fifteen proposer sessions exactly — it reaches 67.3%: the population never improved on its first round's best branch across two further rounds. EvolveNet reaches 70.7% under the same budget (win 14 / lose 9 against the population baseline on paired items). The margin on this homogeneous benchmark is within one noise band, but the mechanism difference is visible in the artifacts: the centralized optimizer's strongest mechanism helps one database family while measurably misleading others, so its budget goes into gating a single mechanism, whereas the collaborating clients develop five specialized mechanism families *concurrently* — and on DS-1000, where that decomposition matters most, the merged harness is 9.0 points above the strongest single client (Sec. 6.4). What this comparison isolates deserves care. EvolveNet differs from the centralized population in two ways at once: its search is decomposed over shards *and* each branch is instructed to specialize. A hypothetical centralized optimizer given the same five-way partition, the same specialist instruction and the same aggregator would perform exactly the computation EvolveNet performs — data locality constrains *where* that computation runs, not what it is. We therefore

read these numbers as evidence that *decomposed specialist search plus evidence-gated aggregation* beats undecomposed pooled search at equal budget, and that EvolveNet preserves this benefit without moving any workload off its client — a constraint the centralized method does not have to satisfy. Distributing the workload is what makes the decomposition necessary and legitimate; it is not independently the source of the gain.

6.7 Does any of this depend on the model?

We repeated the full BIRD pipeline — identical shards, specialist instruction, scope-typed merge, gate, and budget — with the entire model stack (solver, proposer, and merger) swapped from deepseek-v4-flash to MiMo-V2.5, a reasoning model from a different provider. Every qualitative behavior replicates: the round-1 merge is accepted with a large gain, a later harmful merge is rejected by the gate and rolled back, and the final merged harness reaches 72.7% against 72.0% for select-best and 62.7% for the unevolved harness (EvolveNet vs. unevolved: win 23 / lose 8, p=0.011; vs. select-best: +0.7 points, win 10 / lose 9, the same direction and magnitude as with the original stack on this homogeneous benchmark). The qualitative behavior of EvolveNet — accepted gains, gate-rejected regressions, and a final merged harness ahead of both baselines — is therefore not specific to the original model stack.

6.8 Scaling evolutionary search through parallel trajectories

An EvolveNet round costs K parallel local-evolution sessions plus one merger session and one gate measurement. Client phases dominate wall-clock, and because they are independent the speedup over evolving the same programs sequentially scales with the number of clients — bounded above by K, and below it only by the straggler that ends each round. Serializing the identical client sessions of our runs would take 1.93× (K=2), 3.96× (K=5) and 5.37× (K=7) longer than the parallel phase actually took; the gap from the ideal K widens as shards shrink and client runtimes become more uneven — the same shard-size effect that bends the accuracy curve of Fig. 3. End to end, including aggregation and gating, an entire T=3 BIRD run takes 6110 s at K=5 against 1693 s for the per-round centralized optimizer and 3655 s for the equal-budget centralized population. Collaboration therefore does not make a fixed budget cheaper in absolute wall-clock — it makes a *larger* search affordable at bounded serial depth: the K=5 run explores five specialist trajectories plus an aggregation in roughly three and a half times the wall-clock of one per-round centralized run, and reaches 70.7% where the centralized alternatives reach 68.7% and 67.3% (Sec. 6.6). The aggregation step adds one LLM session per round independent of K; gate measurements reuse cached solver calls and add seconds. What crosses the network is a program of a few hundred lines — kilobytes, a payload whose size is set by the harness rather than by the model.

These measurements characterize the dimensions along which EvolveNet scales. The search grows with K while the number of rounds does not: the client phase is bounded by its slowest branch, so K concurrent trajectories cost what the longest one costs (1.9–5.4× below serial). The server's schedule does not lengthen with K either: one merger session and one cached gate measurement per round, whether it aggregates two clients or seven, although the merger's input grows with the number of deltas it reads. Each client transmits a harness-scale payload rather than a model-scale update, so per-client traffic is set by program size, while total server ingress grows approximately linearly with K. And the split is not paid for in accuracy: every multi-client configuration we ran is at or above the single-client baseline (Fig. 3). Adding a deployment therefore adds a search trajectory and a domain of experience to the shared harness without adding a round.

6.9 What the aggregator actually does

The merge reports make the operator concrete. On BIRD, the server received a value-grounding probe hard-coded to one client's database, *generalized* it into a dynamic categorical-value probe verified on three databases, adopted a second client's SQL-dialect auto-fix verbatim into the first client's simpler retry structure, and *rejected* four shard-specific prompt rules by citing the specific graded questions each one broke. On DS-1000, the round-2 merge gated one client's schema pre-analysis to Pandas — reproducing, at the server, the same library gating that a centralized optimizer was forced to discover for itself — while promoting execution-retry logic with cross-library evidence

to global scope. Every adoption decision in these reports cites either a graded per-item verdict or a trace.

7 LIMITATIONS

Data locality is not privacy. Client shards never leave the client — local evolution, candidate selection, and the per-item verdicts are all computed client-side, and what crosses the boundary is the harness source, its diff against the broadcast base, and boolean verdicts. That is a locality guarantee, not a privacy one: a program edit can itself encode client information (our qualitative analysis shows a probe hard-coded to one client's schema), and quantifying such leakage requires a threat model we leave to future work. The method also assumes labeled validation data is available to the server and is representative of the deployment, a stronger requirement than parameter-space aggregation typically makes. Finally, the dispatch key that gates HOME mechanisms must be observable at inference; it is for databases, libraries and repositories, whereas for the LiveCodeBench check it is benchmark difficulty metadata — a limitation of that benchmark as a multi-client testbed.

Artifact growth. Scope-typed adoption adds code monotonically when gates keep passing: over three rounds the BIRD shared harness grew from 14 lines (the unevolved harness) to 253, and the DS-1000 shared harness from 17 to 409. Nothing in the current gate penalizes a merge that ties on accuracy while adding lines. A strict tie-rejecting gate is not the answer — replaying it across our runs forgoes 14 items on DS-1000 and 5 on LiveCodeBench (Appendix F) — but a size- or latency-aware gate, mechanism retirement, and measurements beyond $T{=}3$ rounds are needed before claims about long-horizon accumulation. Our observations cover $T \leq 3$ and $K \leq 7$.

The paired comparison is a counterfactual. Within a round we score the shared harness that each aggregation rule *would have* produced from one run's client snapshot; we do not run $|\text{variants}|$ independent end-to-end runs. This is deliberate — it removes the $\sigma{\approx}3$ client-quality variance that would otherwise dominate — but it means the rows of Table 3 share their client population, and divergence beyond the round in question is not modeled.

One seed per aggregation rule. The aggregation comparison of Table 3 rests on a single snapshot per rule. The ordering it produces is corroborated by the per-library breakdown of Table 2 and by the mechanism-level retention analysis of Sec. 6.5.

Label use. Labels enter the loop through the local training shards, for client-side candidate selection, and through a separate validation slice, for server-side acceptance gating. The method is therefore supervised collaborative adaptation, not the label-free test-time setting of prior harness-evolution work, and its numbers are not comparable to that line.

Ambiguous gold. Within the 40-item hard slice specifically, the audit flags 9 items whose gold is not uniquely determined by the question and its hint (e.g. the question asks for an average while the gold enumerates rows). We keep them for comparability with prior work but they cap the reachable score. All main-text BIRD numbers use this 150-item held-out set.

8 CONCLUSION

Prior harness evolution studies how one agent improves; this paper studies how improvements discovered by many agents accumulate in a shared evolving artifact. We introduced EvolveNet, a collaborative harness-evolution framework in which data-local agent deployments evolve a broadcast harness along parallel specialist trajectories, and a server composes their edits into one shared program through scope-typed, evidence-gated aggregation. Once redistributed, that program lets every deployment inherit the experience the others discovered. Across five evaluation settings the merged harness improves on the unevolved program every client starts from, and in every setting it also improves on retaining the strongest single client. The per-item acceptance gate keeps the committed trajectory non-decreasing on the criterion it measures, and on BIRD EvolveNet avoids the stagnation and rollback that both equal-budget centralized alternatives exhibit. The broader lesson is that

the artifact shared during collaborative agent improvement need not be a parameter vector: an executable program can carry a deployment's operational experience across organizational boundaries, provided its adaptations can be composed and behaviorally validated. Composing such programs is a practical — if fundamentally non-arithmetic — aggregation problem. We expect the scope-typing principle, and the discipline of grounding every adoption in per-item behavioral evidence, to transfer to richer agent ecosystems than the ones studied here.

REFERENCES

Qianshu Cai, Yonggang Zhang, Xianzhang Jia, Huajiang Zheng, Wei Xue, Jun Song, Xinmei Tian, and Yike Guo. MOSS: Self-evolution through source-level rewriting in autonomous agent systems. *arXiv preprint arXiv:2605.22794*, 2026.

Xiang Chen, Yuling Shi, Qizhen Lan, Yuchao Qiu, Min Wang, Xiaodong Gu, and Yanfu Yan. Fed-SE: Federated self-evolution for privacy-constrained multi-environment LLM agents. *arXiv preprint arXiv:2512.08870*, 2025.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning*, 2024.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.

Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *International Conference on Learning Representations, ICLR*, 2024.

Yufei He, Juncheng Liu, Yue Liu, Yibo Li, Tri Cao, Zhiyuan Hu, Xinxing Xu, and Bryan Hooi. EvoTest: Evolutionary test-time learning for self-improving agentic systems. *arXiv preprint arXiv:2510.13220*, 2025.

Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations, ICLR*, 2025.

Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations, ICLR*, 2023.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations, ICLR*, 2025.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations, ICLR*, 2024.

Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank J. Reddi, Sebastian U. Stich, and Ananda Theertha Suresh. SCAFFOLD: Stochastic controlled averaging for federated learning. In *International Conference on Machine Learning, ICML*, 2020.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations, ICLR*, 2024.

Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. DS-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning, ICML*, 2023.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.

Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems, NeurIPS*, 2023.

Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems, MLSys*, 2020.

Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *International Conference on Artificial Intelligence and Statistics, AISTATS*, 2017.

Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, and Bo Han. TTHE: Test-time harness evolution. *arXiv preprint arXiv:2607.08124*, 2026.

Sashank J. Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečný, Sanjiv Kumar, and H. Brendan McMahan. Adaptive federated optimization. In *International Conference on Learning Representations, ICLR*, 2021.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 2024.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems, NeurIPS*, 2023.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations, ICLR*, 2024.

Jingbo Yang, Guanyu Yao, Yang Zhang, Ramana Rao Kompella, Gaowen Liu, and Shiyu Chang. FederatedSkill: Federated learning for agentic skill evolution. *arXiv preprint arXiv:2606.03143*, 2026.

Dixi Yao, Tahseen Rabbani, Manzil Zaheer, and Tian Li. Federation over Text: Insight sharing for multi-agent reasoning. *arXiv preprint arXiv:2604.16778*, 2026.

Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *International Conference on Machine Learning, ICML*, 2024.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation. *arXiv preprint arXiv:2310.02304*, 2023.

Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-Harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025a.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation. In *International Conference on Learning Representations, ICLR*, 2025b.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *International Conference on Learning Representations, ICLR*, 2023.

A Benchmark Construction and Gold Auditing

Per-item gold verification. Every training, validation, and test item passed a benchmark-specific gold check in our execution environment before use. For DS-1000, each problem's reference solution was executed against its own hidden test; 39 of 400 initially sampled problems failed (predominantly Matplotlib, Tensorflow and Pytorch environment sensitivities) and were replaced by same-library problems that pass, topping up from the four largest libraries once the Tensorflow/Pytorch clean pools were exhausted. For SWE-bench Verified, each instance's gold patch was run through the official evaluation harness in our Docker environment; all 80 selected instances resolve. For LiveCodeBench, all 110 selected problems carry non-empty test suites.

BIRD gold audit. BIRD gold queries are known to contain defects. An LLM auditor reviewed every held-out item under a deliberately conservative policy whose default is *accept*; an item is rejected only if (i) the gold SQL fails to execute, (ii) the hint contradicts the question on a concrete value, number, or named entity, or (iii) the gold silently drops a stated condition or adds an unstated one. Difficulty is never grounds for rejection: case mismatches between hint and stored data, discretionary `DISTINCT`, and projection choices are the solver's job and are explicitly accepted. The audit is applied during construction of the held-out set, which contains 150 items: 40 hard and 110 representative. Every BIRD number in this paper is computed on it. We validated the criteria by falsification: an earlier, stricter version rejected 60% of a human-audited slice, and items it flagged were solved at the same rate as items it passed — i.e., it had no discriminative power — which forced the weaker, execution-grounded criteria above.

What the auditor could see, and when it was fixed. We state the audit's provenance precisely. The auditor's entire input is the database schema, the question, the hint, the gold SQL, and the gold query's execution result. It never sees any harness, any generated SQL, or any system's score, so it cannot condition on which method a decision would favor. The criteria and the resulting item list were frozen on 2026-07-22, before any of the end-to-end runs reported here were launched (the earliest, BIRD, completed 2026-07-23); the list was not revisited afterwards. The criteria themselves were developed earlier, and their calibration used a falsification test on solve rates from earlier exploratory runs, so we do not claim the criteria were designed in ignorance of all system behavior — only that the per-item decisions were made without access to system outputs and were fixed before the reported comparisons existed. The audit prompt, the criteria, and the list of excluded items are released with the code. We have not run an independent human audit; obtaining blind human adjudication with inter-annotator agreement would be the stronger protocol and we did not do it.

B Shards and Splits

All splits are drawn with a fixed seed (0) and are disjoint by construction. BIRD: five clients hold 24/18/18/18/22 training items covering card_games, california_schools, formula_1, toxicology, and a mixed shard (debit_card_specializing, superhero, student_club); validation (100) and test (150) are stratified over eleven databases, four of which have no client specialist. DS-1000: five clients hold 20 items each from Pandas, Numpy, Matplotlib, Sklearn, and Scipy; validation and test are stratified over the remaining pool, so Pytorch appears only as a specialist-free domain; Tensorflow items did not survive the gold check and were replaced from the larger libraries, leaving six libraries in the final splits. SWE-bench Verified: five clients hold 30 items across django, sphinx, sympy, astropy, and a pytest+pylint shard, bounded by locally available evaluation images; the test set adds

10 instances from repositories no client trained on. LiveCodeBench: three clients hold 20 items per difficulty band. ClawEval: five clients hold 8/8/4/4/6 training items drawn from the workflow, operations, communication, productivity, and miscellaneous task families, with validation (10) and test (30) allocated from the same families. The K-ablation regroups the BIRD shards (K=2) or splits the mixed shard by database (K=7) with all other settings unchanged.

C PROMPTS

Specialist instruction (abridged). Appended to every client proposer prompt:

> "SPECIALIST MODE — you serve ONE client whose data is the slice in these traces (often a single database). Do NOT aim for a generic harness that works everywhere; aim to become the EXPERT of THIS slice. Study the traces for the recurring, slice-specific failures [...] Encode that specific knowledge — table/column disambiguation, value formats, join paths particular to THIS schema — as durable mechanisms. (Still no hardcoding one question's literal answer; schema- and slice-level knowledge is exactly what you SHOULD capture.) The server will later merge your expertise with other clients' — the more your harness knows that theirs cannot, the more the collaboration gains."

The DS-1000, SWE-bench, and LiveCodeBench variants substitute library, repository, and difficulty-band wording respectively.

Scope-typed adoption rule (abridged). The aggregator receives, in addition to the shared merge scaffold (deltas against the broadcast base, proposal cards, graded verdicts, line budget):

> "SPLIT EVERY CANDIDATE MECHANISM INTO GLOBAL vs HOME-SCOPED before deciding. GLOBAL: mechanisms whose evidence spans databases or whose failure mode is universal [...] adopt ONE best implementation globally. HOME-SCOPED: a rule whose evidence comes from ONE client's home database [...] do NOT adopt it globally, and do NOT reject it for regressing on OTHER databases — adopt it CONDITIONALLY: the harness receives the domain at solve time, so wrap the rule so it only applies on that client's home domain. A home-scoped rule can only change behaviour at home, so off-home regressions are impossible by construction; judge it ONLY on its home evidence. Still REJECT single-question hacks. CONFLICTS DISSOLVE UNDER SCOPING: when two clients changed the same behaviour in opposite directions, keep each side scoped to its own home domain instead of picking a winner."

The holistic, quorum, and conflict-resolution variants replace only this adoption block; scaffold, evidence, and budget are identical across variants.

D MEASUREMENT DETAILS

Two engineering measures make small differences meaningful. All frozen-solver replies are cached on the full request (prompt, system message, temperature, sample count, and repeat index), so two harnesses that ask the same question receive the same answer. Every child process runs with a pinned hash seed: without it, a harness that collects column or table names in a set iterates them in a different order per process, that order reaches the prompt, the cache key changes, and the sampling noise the cache exists to remove reappears. Measured directly, the same frozen harness scored 16/17/17 on three identical evaluations before the seed was pinned; afterwards, two full repeats produced byte-identical SQL on 50/50 items.

Aggregation rules are additionally compared *paired*. Comparing rules by their end-to-end scores confounds a rule with the quality of the clients it happened to receive, and the run-to-run spread in the latter is larger than the effect of interest (Sec. 6.1). We therefore also apply every rule to the same snapshot of client harnesses and report the shared harness each would have produced, so that a difference is attributable to the rule alone. ClawEval is graded on a continuous scale by an independent judge model, so its paired test is a Wilcoxon signed-rank test on per-task score differences rather than an exact McNemar test on binary outcomes.

E KEEPING THE STRONGEST CLIENT, ACROSS ALL FIVE SETTINGS

Sec. 6.4 isolates the aggregation operator on DS-1000, where every rule is applied to one identical client snapshot. The end-to-end comparison against *select-best* — the strongest client by the server's validation slice, promoted without any aggregation — was also run in the four remaining settings, under the same protocol and the same held-out sets as Table 1:

	BIRD	DS-1000	LCB	SWE-V	ClawEval
Select-best client	69.3	59.5	43.3	42.5	70.6
EvolveNet	**70.7**	**68.5**	**66.7**	**57.5**	**74.1**
paired win / lose	9 / 7	33 / 15	7 / 0	8 / 2	17 / 8
paired p	0.80	0.013	0.016	0.11	0.080

EvolveNet leads the end-to-end select-best protocol in all five settings, and the paired counts favour it in each. The tests are exact McNemar on the four binary benchmarks and Wilcoxon signed-rank on ClawEval; combining the five by Fisher's method gives $p = 0.0027$. This comparison contrasts two complete protocols, so it does not by itself isolate the aggregation operator — the identical-snapshot replay of Sec. 6.4 does that. The margin tracks how much the clients' workloads differ: it is widest where shards are drawn along library, difficulty and repository boundaries, and narrowest on BIRD, whose eleven databases share one task shape and whose clients therefore discover overlapping material.

F GATE-RULE REPLAY

Replaying every acceptance decision under a strict-improvement rule ($|\textit{fixed}| > |\textit{broke}|$, ties rejected) leaves the committed artifact unchanged on BIRD and changes it on DS-1000 and LiveCodeBench. For the runs whose strict-replay artifact was itself evaluated on held-out data:

Run	tie-accepting (protocol, %)	strict replay (%)	difference (pts)
BIRD $K{=}5$	70.7	70.7	—
BIRD $K{=}2$	69.3	69.3	—
DS-1000	68.5	61.5	-7.0
LiveCodeBench	66.7	50.0	-16.7

Every decision that differs favours the tie-accepting rule, by 14 and 5 items respectively. Admitting a merge that neither improves nor damages the validation slice preserves the mechanisms it carries into the next round, and the artifacts built on top of it are stronger on held-out data. No replay alters any significance conclusion.